YottaFlex AI — Supply-Chain Traceability & Recall Simulation

Requirements Analysis · v0.1 · 2026-10-30

> **Scope:** Analysis only. No code, no dependency installs, no commits. All assumptions and proposed decisions are explicitly labelled **[ASSUMPTION]** or **[DECISION NEEDED]**.

Table of Contents

1. Functional Requirements
2. Non-Functional Requirements
3. Traceability Matrix
4. Stakeholder Map & SIPOC
5. Business Workflows
6. Entity Relationships & Cardinalities
7. Business Rules & Quantity-Accounting Assumptions
8. Acceptance Criteria & Test Strategy
9. Risks, Ambiguities & 72-Hour Delivery Plan
10. Technology-Stack Recommendation

1. Functional Requirements

FR-01 — Core Entity Modelling

ID	Requirement
FR-01.1	The system MUST implement all eleven prescribed entities: `Supplier`, `MaterialLot`, `Product`, `BOM`, `ProductionOrder`, `FinishedLot`, `Shipment`, `Customer`, `Delivery`, `RecallCase`, `RecallAction`.
FR-01.2	Every entity MUST carry: a primary key in the prescribed format, lifecycle state, effective-date range, and audit timestamps (`created_at`, `updated_at`).
FR-01.3	Foreign keys MUST reference valid parent records that were generated before dependent records.
FR-01.4	Many-to-many relationships MUST be resolved through explicit bridge/junction tables, not arrays inside records.
FR-01.5	Lifecycle states MUST only follow defined allowed transitions (see §7).

FR-02 — ID Formats

ID	Requirement
FR-02.1	Raw-lot IDs: `RAWLOT-<supplier_id>-2026-<6-digit-zero-padded-counter>`.
FR-02.2	Production/work-order IDs: `WO-2026-<6-digit>`.
FR-02.3	Finished-lot IDs: `FINLOT-2026-<6-digit>`.
FR-02.4	Shipment IDs: `SHP-2026-<6-digit>`.
FR-02.5	Recall-case IDs: `REC-2026-<6-digit>`.
FR-02.6	Counters MUST be zero-padded to exactly 6 digits and globally unique within their type.
FR-02.7	IDs MUST be immutable once assigned; they MUST NOT be reused or overwritten.

FR-03 — Dataset Scale & Seed

ID	Requirement
FR-03.1	Full-scale targets: 500 suppliers, 10,000 raw lots, 5,000 production orders, 8,000 finished lots, 20,000 deliveries.
FR-03.2	A **starter mode** with reduced counts (≈ 10 % of full) MUST exist for fast iteration and CI.
FR-03.3	Both modes MUST share identical generation logic; scale is controlled by a single parameter.
FR-03.4	Default random seed is `20261022`; seed MUST be overridable at the CLI.
FR-03.5	Running the generator twice with the same seed MUST produce byte-identical output files.

FR-04 — Supply-Chain Process Simulation

ID	Requirement
FR-04.1	The generator MUST simulate raw-material consumption: link each `ProductionOrder` to one or more `MaterialLot` inputs via a `LotConsumption` bridge table.
FR-04.2	Each `FinishedLot` MUST trace back to its `ProductionOrder` and therefore to its raw-lot inputs.
FR-04.3	Finished lots MUST be packaged into `Shipment` records that reference a `Customer`.
FR-04.4	Each `Shipment` contains one or more `Delivery` line items; total deliveries MUST reach 20,000.
FR-04.5	A `Supplier` MAY supply multiple material types; a `MaterialLot` belongs to exactly one `Supplier`.

FR-05 — Recall Simulation

ID	Requirement
FR-05.1	A `RecallCase` is triggered when a defective `MaterialLot` or `FinishedLot` is identified.
FR-05.2	Downstream genealogy traversal MUST identify all `FinishedLot` records that consumed the affected input lot(s), and all `Delivery` records that shipped them.
FR-05.3	For each impacted delivery, a `RecallAction` record MUST be created (notify, quarantine, recover, or dispose).
FR-05.4	Lifecycle states on impacted `FinishedLot` and `Delivery` records MUST be updated to reflect quarantine/recall status.
FR-05.5	Notification events MUST be persisted as explicit event records; original records MUST NOT be overwritten.
FR-05.6	Recall completion rate MUST be calculable from the ratio of completed `RecallAction` records to total in-scope actions.

FR-06 — Edge-Case Injection

ID	Requirement
FR-06.1	**EC-1 (Orphan Input Lot):** At least one `FinishedLot` MUST reference a `MaterialLot` that does not exist in the dataset.
FR-06.2	**EC-2 (Quantity Overshipment):** At least one `Delivery` MUST have a shipped quantity exceeding the available quantity of its source `FinishedLot`.
FR-06.3	**EC-3 (Missed Customer):** At least one customer who received an impacted lot MUST be absent from the recall notification list.
FR-06.4	**EC-4 (Available Recalled Inventory):** At least one `FinishedLot` subject to a recall MUST retain an `AVAILABLE` lifecycle status, simulating a missed quarantine.
FR-06.5	All four edge cases MUST be labelled in `ground_truth.csv` / `ground_truth.json` with the affected record IDs and edge-case code.

FR-07 — Ground Truth & Detection

ID	Requirement
FR-07.1	The generator MUST export a `ground_truth` file listing every injected anomaly with: anomaly code, affected entity type, affected record ID, and expected detection flag.
FR-07.2	Detection logic MUST NOT read the ground-truth file at runtime; it MUST derive findings independently.
FR-07.3	A separate validation/detection script MUST compare its findings to ground truth and report precision, recall, and F1.

FR-08 — Validation

ID	Requirement
FR-08.1	Primary-key uniqueness across all records of each entity.
FR-08.2	Foreign-key integrity: every FK reference resolves to an existing PK.
FR-08.3	Enum fields contain only allowed values.
FR-08.4	Date ordering: `effective_start` ≤ `effective_end`; `created_at` ≤ `updated_at`; production dates follow material receipt dates.
FR-08.5	Quantity reconciliation: `sum(consumed_qty)` across `LotConsumption` ≤ `MaterialLot.quantity`; `sum(delivered_qty)` per `FinishedLot` ≤ `FinishedLot.quantity` (except EC-2).
FR-08.6	Lifecycle transition validation: e.g., a `QUARANTINED` lot cannot revert to `AVAILABLE` except via an explicit `RELEASED` action.
FR-08.7	No duplicate records within a single entity table.
FR-08.8	Validation MUST run post-generation and report a structured summary (pass/fail per rule, count of violations).

FR-09 — Exports & Documentation

ID	Requirement
FR-09.1	Output: CSV files (one per entity table) and at least one SQL seed script.
FR-09.2	A `README.md` with: project overview, prerequisites, generator commands, seed instructions, and verification steps.
FR-09.3	A `DATA_DICTIONARY.md` documenting every field of every entity (type, nullable, FK, allowed values, description).
FR-09.4	A `SOURCE_MANIFEST.json` recording: generator version, seed, timestamp, row counts per table, and file checksums (SHA-256).
FR-09.5	A `VALIDATION_SUMMARY.md` showing rule-by-rule pass/fail results.
FR-09.6	BPMN 2.0 XML process models for: (a) normal order-to-delivery flow, (b) recall initiation and execution, (c) exception-handling flow.
FR-09.7	An Entity-Relationship Diagram in a diffable text format (e.g., Mermaid or PlantUML).

FR-10 — Reproducibility

ID	Requirement
FR-10.1	The generator MUST be runnable with a single CLI command after setup.
FR-10.2	Two independent runs with seed `20261022` MUST produce identical SHA-256 checksums on all output files.
FR-10.3	The comparison report MUST display row counts and checksums side-by-side for both runs.

2. Non-Functional Requirements

ID	Category	Requirement
NFR-01	**Portability**	Runs on macOS, Linux, and Windows with freely available runtimes (Python 3.11+). No paid APIs, no cloud accounts mandatory.
NFR-02	**Performance**	Full-scale generation completes within 5 minutes on a modern laptop (4-core, 16 GB RAM). Starter-mode generation completes within 30 seconds.
NFR-03	**Security**	No secrets, credentials, or PII in any committed file. Seed files contain only synthetic data.
NFR-04	**Reproducibility**	Deterministic output given the same seed and scale.
NFR-05	**Maintainability**	Code split into clearly named modules (entities, generators, validators, exporters). Each module < 500 lines.
NFR-06	**Testability**	Core functions are unit-testable in isolation (pure functions where possible). CI runs in < 5 min.
NFR-07	**Auditability**	All state changes persisted as immutable event records; no destructive updates to history.
NFR-08	**Observability**	Generator logs progress to stderr (entity type, count, elapsed time).
NFR-09	**Licensing**	All dependencies use OSI-approved open-source licenses.
NFR-10	**Repository hygiene**	`.gitignore` excludes generated artefacts; secrets never committed.

3. Traceability Matrix

Brief Requirement	FR ID(s)	Planned Artefact	Verification Method
11 core entities	FR-01.1	`models/entities.py` (or equivalent schema)	Schema inspection + FR-08.1 uniqueness check
PKs, FKs, cardinalities, dates, lifecycle, bridge tables	FR-01.2–FR-01.5	Entity schema + ERD	FR-08 validation suite
Generation order (parent before child)	FR-01.3	Generator orchestrator	FK integrity check (FR-08.2)
Prescribed ID formats	FR-02.1–FR-02.7	ID factory module	Regex-based validation test
Full dataset scale targets	FR-03.1	Generator config	Row-count assertion in test suite
Starter mode	FR-03.2–FR-03.3	`--mode starter` CLI flag	Run starter mode; check row counts ≈ 10 %
Fixed seed 20261022	FR-03.4–FR-03.5	`--seed` CLI arg	Determinism test (FR-10.2)
Raw-material consumption	FR-04.1–FR-04.2	`LotConsumption` bridge table	Quantity reconciliation (FR-08.5)
Shipments & deliveries	FR-04.3–FR-04.4	`Shipment`, `Delivery` generators	Row-count + FK checks
Recall trigger & genealogy	FR-05.1–FR-05.2	Recall engine module	Upstream/downstream traversal test
RecallAction creation	FR-05.3–FR-05.5	Recall engine + event log	Ratio completeness check
Recall completion metric	FR-05.6	KPI reporter	Compare expected vs. actual ratio
4 labelled edge cases	FR-06.1–FR-06.5	Edge-case injector module	Detection script vs. ground truth (FR-07.3)
Ground-truth file	FR-07.1–FR-07.3	`ground_truth.csv`	Precision/recall/F1 report
8-rule validation suite	FR-08.1–FR-08.8	`validators/` module	Validation summary; CI gate
CSV/SQL export	FR-09.1	Exporter module	File existence + checksum

Brief Requirement	FR ID(s)	Planned Artefact	Verification Method
README, data dict, manifest	FR-09.2–FR-09.4	`README.md`, `DATA DICTIONARY.md`, `SOURCE_MANIFEST.json`	Content review + automated field-count check
Validation summary	FR-09.5	`VALIDATION_SUMMARY.md`	CI artifact
BPMN 2.0 process models	FR-09.6	`docs/bpmn/*.bpmn`	XML well-formedness + peer review
ERD	FR-09.7	`docs/erd.mmd` or `.puml`	Mermaid render check
Single-command run	FR-10.1	`Makefile` + `pyproject.toml`	Smoke test in CI
Reproducibility (2× run = same checksum)	FR-10.2–FR-10.3	`scripts/verify_reproducibility.sh`	Automated SHA-256 comparison
BPM artefacts (SIPOC, stakeholder, rules, KPIs)	—	`docs/bpm/`	Peer review

4. Stakeholder Map & SIPOC

4.1 Stakeholder Map

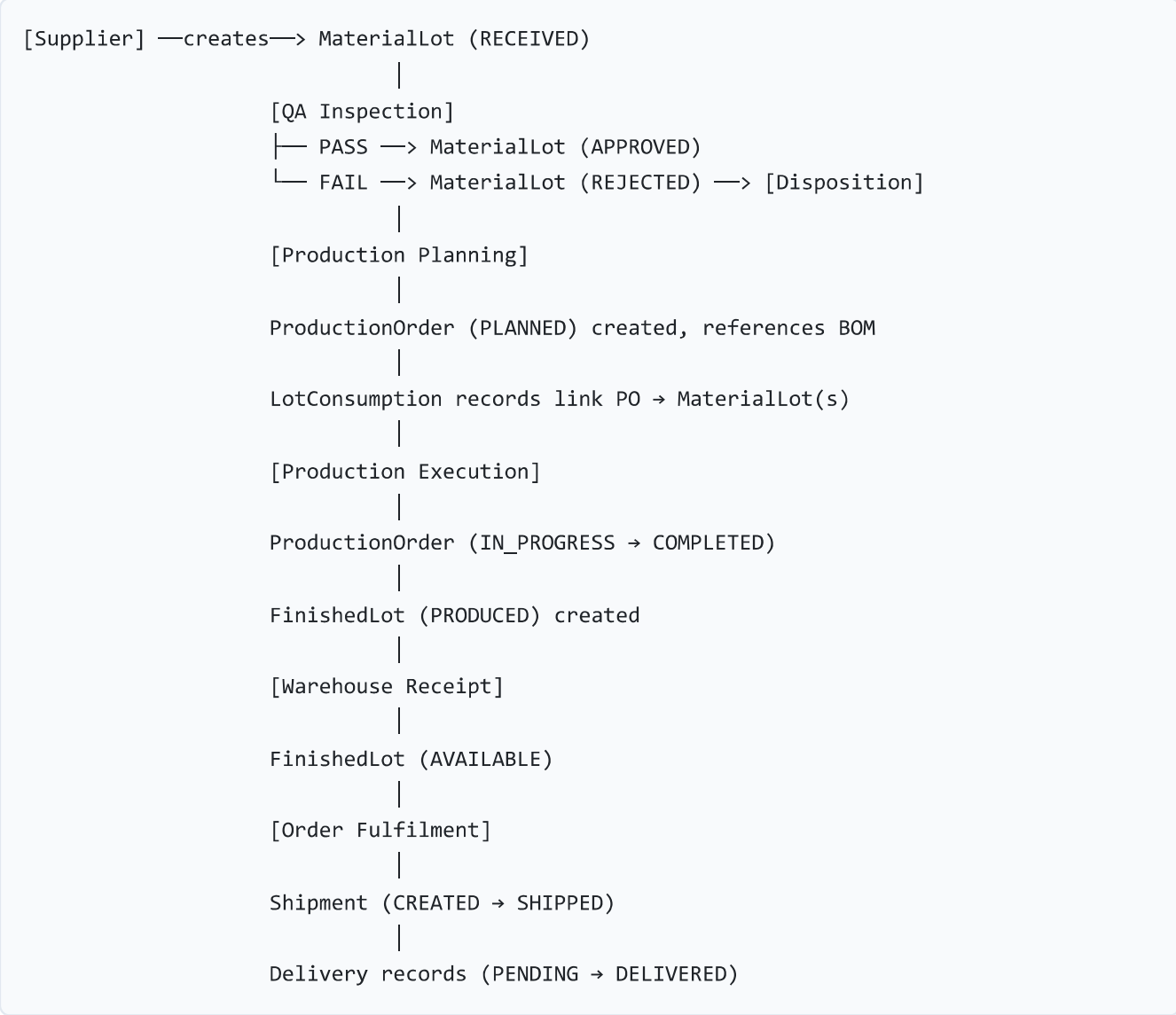
Stakeholder	Role	Interests / Concerns
Supplier	External	Lot provenance, compliance certificates, recall liability
Production Manager	Internal operator	BOM accuracy, work-order completion, scrap tracking
Quality Assurance (QA)	Internal authority	Defect detection, recall trigger authority, disposition decisions
Warehouse / Logistics	Internal operator	Inventory availability, quarantine zones, shipment scheduling
Customer	External recipient	Delivery accuracy, recall notifications, replacement fulfilment
Regulatory Body	External authority	Traceability completeness, recall reporting timelines
IT / Data Engineering	Internal enabler	Data pipeline reliability, reproducibility, schema integrity
Business Analyst / Challenge Evaluator	External assessor	Edge-case coverage, KPI measurability, documentation quality

4.2 SIPOC

	Suppliers	Inputs	Process	Outputs	Customers
S	Raw-material vendors	Purchase orders	Receive & inspect raw materials	Accepted/rejected `MaterialLot`	Production
I	Approved `MaterialLot` records, BOM	Work orders	Manufacture & assemble	`ProductionOrder`, `FinishedLot`	Warehouse
P	Finished lots, shipment orders	Pick-and-pack instructions	Pack, ship, deliver	`Shipment`, `Delivery`	End customers
O	Defect reports, recall trigger	Impacted-lot list	Execute recall	`RecallCase`, `RecallAction`, notifications	QA, Regulatory, Customer
C	QA, Regulatory	Validation results	Validate & report	`VALIDATION_SUMMARY`, `ground_truth` diff	Evaluator / Business Analyst

5. Business Workflows

5.1 Normal Flow — Order to Delivery



5.2 Recall Flow — Initiation to Closure

```
[Trigger: Defect detected on MaterialLot or FinishedLot]
          │
    RecallCase (OPEN) created by QA
          │
    [Downstream Genealogy Traversal]
    - Find all FinishedLots that consumed affected MaterialLot(s)
    - Find all Deliveries that shipped impacted FinishedLots
          │
    [Scope Determination]
    - RecallCase.scope_lot_ids[] populated
    - RecallCase.scope_delivery_ids[] populated
          │
    For each impacted Delivery:
    ├── RecallAction (NOTIFY) created → CustomerNotification event persisted
    ├── Customer acknowledges → RecallAction (QUARANTINE) → FinishedLot status →
QUARANTINED
    ├── Physical recovery → RecallAction (RECOVER)
    └── QA decides disposition → RecallAction (DISPOSE or RELEASE)
          │
    [Completion Check]
    completion_rate = COUNT(RecallAction.status = COMPLETED) / COUNT(all RecallActions for
case)
          │
    If completion_rate = 1.0 → RecallCase (CLOSED)
    Else → RecallCase remains OPEN with outstanding actions listed
```

5.3 Exception Paths

Exception	Trigger	Handling
EC-1: Orphan Input Lot	FK resolution fails during genealogy traversal	Log `MISSING_INPUT_LOT` event; flag FinishedLot as `DATA_INTEGRITY_ERROR`; continue traversal with known lots
EC-2: Quantity Overshipment	`sum(delivered qty) > FinishedLot.quantity`	Log `QUANTITY_EXCEEDED` validation error; Delivery remains; flag in validation summary
EC-3: Missed Customer	Customer in scope but absent from RecallAction list	Detected by comparing `scope_delivery_ids` to `RecallAction.delivery_id` set; logged as `MISSED_NOTIFICATION`
EC-4: Available Recalled Inventory	FinishedLot in recall scope retains `AVAILABLE` status	Detected by querying: FinishedLot IN recall scope AND status = `AVAILABLE`; logged as `QUARANTINE_FAILURE`
Rejected MaterialLot consumed	`MaterialLot.status = REJECTED` referenced in `LotConsumption`	Validation error `REJECTED_LOT_CONSUMED`; not an injected edge case but must be caught
Shipment without Delivery	`Shipment` with no associated `Delivery` records	Validation warning `EMPTY_SHIPMENT`

6. Entity Relationships & Cardinalities

6.1 Entity Summary

Entity	PK Format	Key FKs	Lifecycle States
`Supplier`	`SUP-<6-digit>`	—	`ACTIVE`, `SUSPENDED`, `INACTIVE`
`MaterialLot`	`RAWLOT-<sup id>-2026-<6-digit>`	`supplier_id` → Supplier	`RECEIVED`, `APPROVED`, `REJECTED`, `CONSUMED`, `QUARANTINED`, `DISPOSED`
`Product`	`PROD-<6-digit>`	—	`ACTIVE`, `DISCONTINUED`
`BOM`	`BOM-<6-digit>`	`product_id` → Product	`DRAFT`, `APPROVED`, `SUPERSEDED`
`BOMLine` *(bridge)*	composite `(bom_id, material_type, line_no)`	`bom_id` → BOM	—
`ProductionOrder`	`WO-2026-<6-digit>`	`product_id`, `bom_id`	`PLANNED`, `IN_PROGRESS`, `COMPLETED`, `CANCELLED`
`LotConsumption` *(bridge)*	composite `(wo_id, raw_lot_id)`	`wo_id` → ProductionOrder, `raw_lot_id` → MaterialLot	—
`FinishedLot`	`FINLOT-2026-<6-digit>`	`wo_id` → ProductionOrder	`PRODUCED`, `AVAILABLE`, `SHIPPED`, `QUARANTINED`, `DISPOSED`, `RELEASED`
`Shipment`	`SHP-2026-<6-digit>`	`customer_id` → Customer	`CREATED`, `SHIPPED`, `DELIVERED`, `PARTIALLY_RECALLED`, `RECALLED`
`Customer`	`CUST-<6-digit>`	—	`ACTIVE`, `INACTIVE`
`Delivery`	`DEL-2026-<6-digit>`	`shipment_id` → Shipment, `fin_lot_id` → FinishedLot	`PENDING`, `DELIVERED`, `RECALLED`, `QUARANTINED`
`RecallCase`	`REC-2026-<6-digit>`	`trigger_lot_id` (MaterialLot or FinishedLot)	`OPEN`, `IN PROGRESS`, `CLOSED`, `CANCELLED`
`RecallAction`	`RACT-2026-<6-digit>`	`recall_id` → RecallCase, `delivery_id` → Delivery	`NOTIFIED`, `QUARANTINE_REQUESTED`, `RECOVERED`, `DISPOSED`, `RELEASED`, `COMPLETED`

Entity	PK Format	Key FKs	Lifecycle States
`EventLog` *(audit)*	`EVT-<6-digit>`	polymorphic `(entity_type, entity_id)`	— (append-only)

6.2 Cardinalities

```
Supplier         1 ──< MaterialLot          (one supplier → many lots)
Product          1 ──< BOM                  (one product → many BOM versions)
BOM              1 ──< BOMLine              (one BOM → many lines)
ProductionOrder  1 ──< LotConsumption       (one PO → many consumption records)
MaterialLot      1 ──< LotConsumption       (one raw lot consumed across ≥1 POs — partial
qty)
ProductionOrder  1 ──< FinishedLot          (one PO → many finished lots)
Shipment         1 ──< Delivery             (one shipment → many delivery lines)
FinishedLot      1 ──< Delivery             (one finished lot → many deliveries, guarded
by qty)
Customer         1 ──< Shipment             (one customer → many shipments)
RecallCase       1 ──< RecallAction         (one recall case → many actions)
Delivery         1 ──< RecallAction         (one delivery → at most one action per recall
case)
MaterialLot      0..1 ── RecallCase         (optional trigger source)
FinishedLot      0..1 ── RecallCase         (optional trigger source)
```

6.3 ERD (Mermaid — to be refined)

7. Business Rules & Quantity-Accounting Assumptions

7.1 Quantity Accounting

Rule ID	Rule	[ASSUMPTION / DECISION NEEDED]
QA-01	`SUM(LotConsumption.consumed_qty WHERE raw_lot_id = X)` ≤ `MaterialLot[X].quantity`	**[ASSUMPTION]** Partial consumption is allowed (a lot can be split across POs up to its quantity).
QA-02	`SUM(Delivery.shipped_qty WHERE fin_lot_id = Y)` ≤ `FinishedLot[Y].quantity` — except EC-2	**[ASSUMPTION]** A finished lot can be split across multiple deliveries/shipments.
QA-03	EC-2 violation is intentional; exactly one delivery per run exceeds source lot quantity.	**[DECISION NEEDED]** Should EC-2 apply to one delivery or a configurable number?
QA-04	`FinishedLot.quantity` = f(BOM yield ratio, raw inputs consumed). Yield ratio range?	**[DECISION NEEDED]** Assume yield ratio uniform [0.85, 0.98]? Or fixed per Product?
QA-05	Units of measure: assume all quantities in same UoM (integer units) for simplicity.	**[ASSUMPTION]** Mixed UoM handling is out of scope for v1.

7.2 Lifecycle Transition Rules

Entity	Allowed Transitions
`MaterialLot`	`RECEIVED` → `APPROVED` or `REJECTED`; `APPROVED` → `CONSUMED` or `QUARANTINED`; `QUARANTINED` → `DISPOSED` or `RELEASED` (back to `APPROVED`); no other transitions
`ProductionOrder`	`PLANNED` → `IN_PROGRESS` → `COMPLETED` or `CANCELLED`
`FinishedLot`	`PRODUCED` → `AVAILABLE` → `SHIPPED` or `QUARANTINED`; `QUARANTINED` → `DISPOSED` or `RELEASED`; `RELEASED` → `AVAILABLE`
`RecallCase`	`OPEN` → `IN_PROGRESS` → `CLOSED` or `CANCELLED`
`RecallAction`	`NOTIFIED` → `QUARANTINE_REQUESTED` → `RECOVERED` → `DISPOSED` or `RELEASED`; terminal: `COMPLETED`

7.3 BOM & Production Rules

Rule ID	Rule
BR-01	Every `ProductionOrder` references exactly one approved `BOM`.
BR-02	A `BOM` has at least one `BOMLine`; each `BOMLine` references a material type (not a specific lot).
BR-03	Lot assignment to a PO resolves at generation time: pick lots of the right material type with sufficient remaining quantity.
BR-04	**[ASSUMPTION]** A `BOM` version is fixed at PO creation time; later BOM revisions do not retroactively affect existing POs.
BR-05	**[DECISION NEEDED]** Can a `ProductionOrder` produce more than one `FinishedLot`? (Assumed: YES, 1–5 finished lots per PO.)

7.4 Recall Rules

Rule ID	Rule
RR-01	Recall scope is determined by forward genealogy: affected raw lot → POs that consumed it → finished lots → deliveries.
RR-02	A `RecallCase` against a `FinishedLot` directly scopes to deliveries of that lot (no upstream traversal needed, but upstream tagging is informational).
RR-03	**[ASSUMPTION]** A single delivery is targeted by at most one `RecallAction` per `RecallCase`.
RR-04	EC-3 (missed customer) is injected by excluding exactly one impacted delivery from the recall-action list at generation time.
RR-05	EC-4 (available recalled inventory) is injected by leaving at least one impacted `FinishedLot` in `AVAILABLE` state after recall actions are generated.
RR-06	**[DECISION NEEDED]** Should partial-lot recalls (only some units returned) be modelled? (Proposed: no, for v1.)

8. Acceptance Criteria & Test Strategy

8.1 Acceptance Criteria

AC ID	Criterion	Pass Condition
AC-01	All 11 entities present	Schema inspection finds all tables/classes
AC-02	ID format compliance	Regex test passes for 100 % of records in each entity
AC-03	Full-scale row counts met	Rows: suppliers ≥ 500, raw lots ≥ 10 000, POs ≥ 5 000, fin lots ≥ 8 000, deliveries ≥ 20 000
AC-04	PK uniqueness	Zero duplicate PKs across all tables
AC-05	FK integrity	Zero unresolved FK references (except EC-1 orphan, which is labelled)
AC-06	Quantity accounting	Zero QA-01/QA-02 violations outside EC-2
AC-07	Exactly 4 edge cases injected and labelled	ground_truth contains exactly 4 records (one per EC code)
AC-08	Detection script achieves F1 = 1.0 on ground truth	All 4 edge cases detected, zero false positives
AC-09	Determinism	SHA-256 checksums of run-1 and run-2 outputs are identical
AC-10	Generation performance	Full mode: ≤ 5 min; starter mode: ≤ 30 s
AC-11	Validation summary generated	`VALIDATION_SUMMARY.md` present with all 8 rules reported
AC-12	BPMN files valid XML	`xmllint` / BPMN parser reports 0 errors
AC-13	No secrets in repo	`git-secrets` or `trufflehog` scan reports 0 findings
AC-14	Single-command reproducible run	`make generate SEED=20261022 MODE=full` completes without error

8.2 Test Strategy

```
Layer 1 — Unit Tests (pytest)
  ├── ID factory: test all formats, counter padding, immutability guard
  ├── Lifecycle FSM: test all valid and invalid transitions
  ├── Quantity validator: test normal cases + EC-1/EC-2 scenarios
  ├── Recall genealogy: small synthetic graph, verify correct scope
  └── Edge-case injector: verify exactly one record per EC code

Layer 2 — Integration Tests (pytest)
  ├── Run generator in starter mode (seed=20261022)
  ├── Assert row counts, PK uniqueness, FK integrity
  ├── Run detection script; compare against ground_truth
  └── Assert detection F1 = 1.0

Layer 3 — Determinism Test
  ├── Run generator twice with same seed
  ├── Compute SHA-256 of each output file
  └── Assert all checksums match

Layer 4 — Performance Smoke Test
  ├── Run generator in full mode with timeout
  └── Assert completion within 5 minutes

Layer 5 — Documentation / Artefact Tests
  ├── Assert all required files exist (README, DATA_DICTIONARY, BPMN files, ERD)
  ├── BPMN XML well-formedness check
  └── SOURCE_MANIFEST field-completeness check
```

CI Configuration: GitHub Actions workflow running Layers 1–3 and 5 on every push. Layer 4 (full-mode) runs on tagged releases only (to keep CI under 10 min).

9. Risks, Ambiguities & 72-Hour Delivery Plan

9.1 Risks

Risk ID	Risk	Probability	Impact	Mitigation
R-01	Determinism broken by floating-point or dict ordering	Medium	High	Use integer quantities; sort all collections before serialization; seed all RNG calls explicitly
R-02	Memory pressure at full scale (10 k lots × 20 k deliveries in RAM)	Medium	Medium	Stream CSV writes; use generators, not lists, for large collections
R-03	EC-2 overshipment accidentally masked by quantity reconciliation fix	Medium	High	Inject edge cases after validation generation; guard with explicit skip-flag
R-04	Genealogy traversal too slow on full dataset	Low	Medium	Build adjacency dict indexed by lot ID at generation time (O(n)); avoid $O(n^2)$ scans
R-05	BPMN tooling unfamiliar; models incorrect	Medium	Low	Use bpmn.io online editor (free); validate XML with `xmllint`
R-06	Scope creep (mixed UoM, partial recalls, multi-product BOM)	High	Medium	Explicitly out-of-scope in README; document as next steps
R-07	Git commit history too sparse for evaluators	Low	Medium	Commit after each logical layer (schema → generators → validators → exporters → tests → docs)

9.2 Unresolved Ambiguities

Amb ID	Ambiguity	Proposed Resolution	**[DECISION NEEDED BY OWNER]**
AMB-01	Supplier ID format not prescribed	**Propose:** `SUP-<6-digit>` embedded in RAWLOT prefix	✅ If acceptable, no decision needed
AMB-02	Are `RecallAction` IDs prescribed?	**Propose:** `RACT-2026-<6-digit>`	✅ Confirm or override
AMB-03	Number of recalls to simulate	**Propose:** 5–10 recall cases; 1 primary injected, rest incidental	✅ Confirm count
AMB-04	Does `Delivery` have a quantity field, or is it whole-lot?	**Propose:** `Delivery.shipped_qty` (integer units, subset of FinishedLot.quantity)	✅ Confirm granularity
AMB-05	Does EC-1 (missing input lot) mean the RAWLOT ID is entirely absent from the dataset, or just not linked?	**Propose:** ID is syntactically valid but no matching record exists in MaterialLot table	✅ Confirm interpretation
AMB-06	Should the BPMN models be separate files per process or one combined file?	**Propose:** Three separate `.bpmn` files for clarity	✅ Confirm
AMB-07	Expected recall notification method (email, SMS, EDI)?	**Propose:** Simulated with a `notification_channel` enum field; no real messages	✅ Confirm
AMB-08	Are shipments direct supplier-to-customer or via warehouses?	**Propose:** Manufacturer-to-customer (warehouse is an internal state, not an entity FK)	✅ Confirm or add Warehouse entity

9.3 Realistic 72-Hour Delivery Plan

> **[ASSUMPTION]** Working in approx. 4–6 hour focused blocks. Times are relative to start (T+0 = kick-off after owner review of this document).

```
T+00h  ── BLOCK 0: Design Lock (2 h)
          Owner reviews and resolves all [DECISION NEEDED] items
          Finalise entity schema, enums, lifecycle FSMs
          Commit: docs/schema.md, docs/decisions.md

T+02h  ── BLOCK 1: Project Scaffold & Core Entities (4 h)
          pyproject.toml, directory structure, Makefile
          models/: dataclasses/Pydantic models for all 11 entities + bridges
          Unit tests for ID factory and lifecycle FSM
          Commit: "feat: project scaffold and entity models"

T+06h  ── BLOCK 2: Data Generators — Master Data (4 h)
          generators/suppliers.py, products.py, boms.py, customers.py
          Seeded RNG wrapper; starter/full mode switch
          Unit + integration tests: row counts, FK validity
          Commit: "feat: master data generators"

T+10h  ── BLOCK 3: Data Generators — Transactional Data (6 h)
          generators/material_lots.py, production_orders.py,
            lot_consumption.py, finished_lots.py
          Quantity accounting logic
          Commit: "feat: transactional generators (lots, POs, consumption)"

T+16h  ── BLOCK 4: Shipments, Deliveries & Recall Engine (6 h)
          generators/shipments.py, deliveries.py
          recall/engine.py: genealogy traversal, RecallCase, RecallAction
          Event log (append-only)
          Commit: "feat: shipment, delivery, and recall engine"

T+22h  ── BLOCK 5: Edge-Case Injector & Ground Truth (3 h)
          injectors/edge_cases.py: inject EC-1 through EC-4
          exporters/ground_truth.py: write ground_truth.csv
          Commit: "feat: edge-case injection and ground truth export"

T+25h  ── BLOCK 6: Validation Suite (4 h)
          validators/: PK uniqueness, FK integrity, enums, dates,
            quantities, lifecycle, duplicates
          Validation summary reporter
          Commit: "feat: full validation suite"

T+29h  ── BLOCK 7: Exporters & Determinism Tests (3 h)
          exporters/csv_export.py, sql_seed.py, source_manifest.py
          scripts/verify_reproducibility.sh
          Determinism test (Layer 3)
          Commit: "feat: exporters and reproducibility verification"
```

```
T+32h  ── BLOCK 8: BPM Documentation & BPMN Models (4 h)
          docs/bpmn/: 3 BPMN XML files (draw via bpmn.io, export XML)
          docs/erd.mmd (Mermaid ERD)
          DATA_DICTIONARY.md, VALIDATION_SUMMARY template
          Commit: "docs: BPMN process models, ERD, data dictionary"

T+36h  ── BLOCK 9: CI, README & Final Polish (4 h)
          .github/workflows/ci.yml
          README.md with full run instructions
          AI usage disclosure, assumptions doc
          Commit: "docs: README, CI, AI usage disclosure"

T+40h  ── BLOCK 10: Full-Mode Performance Run & Buffer (4 h)
          Run full mode; verify ≤ 5 min, all AC pass
          Fix any discovered issues
          Commit: "test: full-scale validation and performance sign-off"

T+44h  ── BUFFER (28 h)
          Defect fixes, polish, stretch goals (SQL seed, KPI dashboard)
          Final submission review

T+72h  ── SUBMISSION DEADLINE
```

10. Technology-Stack Recommendation

> **Principle:** minimum viable stack — every dependency must earn its place.

Layer	Tool / Library	Version	Justification	Alternatives Considered
Language	Python	3.11+	Universal data-science ecosystem; native `random.seed()` determinism; evaluator-friendly	—
Data Modelling	`dataclasses` (stdlib) or `pydantic` v2	stdlib / 2.x	Schema enforcement + JSON serialisation; Pydantic gives runtime type validation at near-zero cost	`attrs` (less familiar)
Random generation	`random` (stdlib) + `faker`	stdlib / faker 26	`random.seed()` is deterministic; Faker for realistic names/addresses	`numpy.random` (heavier)
DataFrame manipulation	`pandas`	2.x	Efficient CSV I/O, group-by for quantity checks, merge for FK validation	`polars` (faster, less familiar)
CSV / JSON export	`csv` + `json` (stdlib)	stdlib	No extra dependency; deterministic write order	—
SQL seed export	`sqlite3` (stdlib)	stdlib	Zero-install local SQL; seed script runnable anywhere	PostgreSQL (requires server)
Testing	`pytest`	7+	Industry standard; parametrized tests for edge cases	`unittest` (more verbose)
CLI	`argparse` (stdlib)	stdlib	No extra dep; `--seed`, `--mode`, `--output-dir` flags	`click` (nicer UX, one extra dep)
Build / task runner	`Makefile`	system	Universal; `make generate`, `make test`, `make verify`	`tox`, `just`
BPMN authoring	bpmn.io (browser, free)	online	Free, no install; exports valid BPMN 2.0 XML	Camunda Modeler (desktop, free)
ERD / diagrams	Mermaid (in Markdown)	—	Renders in GitHub natively; diffable text	PlantUML (requires Java)
CI	GitHub Actions	—	Free for public repos; YAML-configured	GitLab CI (if preferred)
Secret scanning	`trufflehog` or `git-secrets`	—	Lightweight; runs in CI as a pre-commit gate	—

What is intentionally excluded:

- No cloud database (all persistence is file-based CSV/SQLite)

- No paid API (Faker generates all synthetic data locally)
- No Docker (Python venv sufficient; Docker adds complexity without benefit here)
- No ORM (SQLAlchemy) — sqlite3 stdlib is sufficient for seed-file generation

Appendix A — Open Decisions Summary

#	Decision	Proposed Default	Owner Sign-off
D-01	EC-2 applies to exactly 1 delivery	1	☐
D-02	Yield ratio range [0.85, 0.98] uniform random	Yes	☐
D-03	1–5 finished lots per production order	Yes	☐
D-04	Supplier ID format: `SUP-<6-digit>`	Yes	☐
D-05	RecallAction ID: `RACT-2026-<6-digit>`	Yes	☐
D-06	5–10 recall cases total per full run	Yes	☐
D-07	Delivery has `shipped_qty` integer field	Yes	☐
D-08	EC-1 = syntactically valid RAWLOT ID, no matching record	Yes	☐
D-09	Three separate BPMN files	Yes	☐
D-10	Notification simulated as enum field (no real messages)	Yes	☐
D-11	No Warehouse entity (warehouse = internal state only)	Yes	☐
D-12	Partial recalls out of scope for v1	Yes	☐

Document prepared by: Antigravity AI Assistant — Requirements Analysis role
AI tool disclosure: This document was generated with AI assistance (Google Antigravity / Gemini). All requirements, ambiguities, and decisions were derived from the provided brief; no external data sources were consulted. All assumptions are explicitly labelled.